



DataFrame no Pandas: Colunas, Filtros, CSV, Pickle e Vetorização vs Laço

Criando vetores básicos com NumPy

Para construir uma tabela, começamos criando vetores de dados. Em Python, esses vetores são representados por **arrays NumPy**. Cada vetor corresponderá a uma futura coluna da tabela.

```
import numpy as np

weight = np.array([60, 72, 57, 90, 95, 72])
height = np.array([1.75, 1.80, 1.65, 1.90, 1.74, 1.91])
subject = np.array(["A", "B", "C", "D", "E", "F"])

print(weight)
print(height)
print(subject)
```

Saída:

[60 72 57 90 95 72]

[1.75 1.8 1.65 1.9 1.74 1.91]

['A' 'B' 'C' 'D' 'E' 'F']

Criando um DataFrame e visualizando

Os vetores são combinados em uma estrutura tabular usando `pd.DataFrame`. Cada vetor se torna uma coluna com um nome específico. O método `head()` exibe as primeiras linhas da tabela.

```
import pandas as pd
```

```
d = pd.DataFrame({  
    "weight": weight,  
    "height": height,  
    "subject": subject  
})
```

```
print(d.head())
```

Saída:

	weight	height	subject
0	60	1.75	A
1	72	1.80	B
2	57	1.65	C
3	90	1.90	D
4	95	1.74	E

Adicionando uma coluna calculada

Identificar colunas existentes

Usamos `weight` e `height` já presentes no DataFrame

Criar operação matemática

Aplicamos a fórmula do IMC: peso dividido pela altura ao quadrado

Atribuir nova coluna

O resultado é aplicado automaticamente a todas as linhas

Podemos criar novas colunas a partir de cálculos entre colunas existentes. A operação é aplicada a todas as linhas — isso é chamado de **operação vetorizada**.

```
d["bmi"] = d["weight"] / (d["height"] ** 2)
print(d.head())
```

Saída: O DataFrame agora contém a coluna `bmi` com o IMC calculado para cada registro.

Removendo uma coluna do DataFrame

Uma coluna pode ser removida quando não é mais necessária. Em pandas usamos o método `drop` com o parâmetro `columns`.

```
d = d.drop(columns=["subject"])  
print(d.head())
```

📄 **Resultado:** A coluna "subject" foi removida. O DataFrame agora contém apenas weight, height e bmi.

Importando dados de um arquivo CSV

Arquivos CSV armazenam tabelas em formato texto simples. Em pandas, usamos `read_csv` para carregar dados de arquivos locais ou URLs remotas.

```
import pandas as pd

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data"
wine = pd.read_csv(url, header=None)
print(wine.head())
```

Wine (UCI): 178 vinhos, 13 medidas químicas + classe. O arquivo não tem cabeçalho, por isso `header=None`; as colunas recebem os nomes 0, 1, 2...

Flexibilidade

Carregue dados de arquivos locais ou URLs

Automático

Pandas detecta a estrutura dos dados automaticamente

Configurável

Controle cabeçalhos, separadores e tipos de dados

Persistência de dados em Python (pickle)

Em Python, usamos **pickle** para salvar e restaurar DataFrames completos. Ele preserva tipos de dados, índices e toda a estrutura do objeto — o formato nativo para persistir objetos pandas.

```
wine.to_pickle("wine.pkl")
del wine
wine = pd.read_pickle("wine.pkl")
print(wine.head(3))
```

Salvar

`to_pickle()`

Armazenar

Arquivo .pkl

Restaurar

`read_pickle()`

⚠️ ⚠️ Só abra arquivos pickle que você mesmo gerou: carregar um pickle de origem desconhecida pode executar código malicioso. Para compartilhar ou arquivar tabelas, prefira CSV ou Parquet (`df.to_parquet`).

Exportando dados para CSV

CSV é o formato universal para trocar dados entre plataformas. Em pandas usamos `to_csv` para salvar o DataFrame, com opção de incluir ou não o índice.

```
wine.to_csv("wine.csv", index=False)
```

📄 **Resultado:** O arquivo `wine.csv` é criado no diretório do projeto, pronto para Excel, Google Sheets ou qualquer ferramenta de análise.

Compatibilidade universal

Abra em qualquer software de análise de dados

Formato legível

Texto simples, fácil de visualizar e editar

Compartilhamento fácil

Ideal para colaboração entre equipes

Filtrando linhas com máscara booleana

Uma condição lógica gera um vetor de `True` e `False`. Apenas as linhas marcadas como `True` são mantidas no resultado.

```
mask = d["height"] > 1.7
print(mask)
filtered = d[mask]
print(filtered)
```

Máscara booleana:

```
0    True
1    True
2   False
3    True
4    True
5    True
Name: height, dtype: bool
```

DataFrame filtrado:

Apenas as linhas onde `height > 1.7` são mantidas. As linhas 0, 1, 3, 4 e 5 aparecem no resultado.

Operações vetorizadas e desempenho

Velocidade

Cálculos sobre colunas inteiras são extremamente rápidos

Cálculos feitos diretamente sobre colunas inteiras são muito rápidos. Isso ocorre porque pandas e NumPy usam **operações vetorizadas** — todos os valores são processados de uma só vez, sem loops Python.

Otimização

NumPy e pandas usam código compilado em C

Processamento em lote

Todos os valores processados simultaneamente

```
import numpy as np
import pandas as pd
from time import perf_counter
# Gera alturas e pesos aleatórios
rheight = np.random.normal(1.8, 0.2, 100_000)
rweight = np.random.normal(72, 15, 100_000)
t0 = perf_counter()
hw = pd.DataFrame({"height": rheight, "weight": rweight})
hw["bmi"] = hw["weight"] / (hw["height"] ** 2)
t1 = perf_counter()
print("Tempo vetorizado:", t1 - t0)
```

Saída típica: Tempo vetorizado: **0.0021 segundos** para 100.000 linhas!

Tipos de dados das colunas (dtype)

Em pandas, cada coluna tem um **tipo interno** chamado dtype. O tipo influencia desempenho, memória e comportamento das operações.

```
print(hw.dtypes)
```

Saída (exemplo):

```
height float64
weight float64
bmi     float64
dtype: object
```

float64

Ponto flutuante de dupla precisão (~15–16 dígitos significativos)

int64

Números inteiros

object

Texto ou dados mistos

bool

Valores True/False

Antipadrão: calcular com loop no DataFrame

Atribuir valores linha a linha é **muito lento**. Prefira sempre operações vetorizadas.

```
from time import perf_counter
import numpy as np
import pandas as pd

rheight = np.random.normal(1.8, 0.2, 100_000)
rweight = np.random.normal(72, 15, 100_000)

t0 = perf_counter()
hw = pd.DataFrame({"height": rheight, "weight": rweight})
hw["bmi"] = np.nan
for i in range(len(hw)):
    hw.loc[i, "bmi"] = hw.loc[i, "weight"] / (hw.loc[i, "height"] ** 2)
t1 = perf_counter()

print("Tempo com loop:", t1 - t0)
```

Evite loops

Iteração linha a linha é extremamente lenta

Use vetorização

Operações em colunas inteiras são muito mais rápidas

Saída típica: Tempo com loop: 8.3 segundos — **3900x mais lento** que a versão vetorizada!

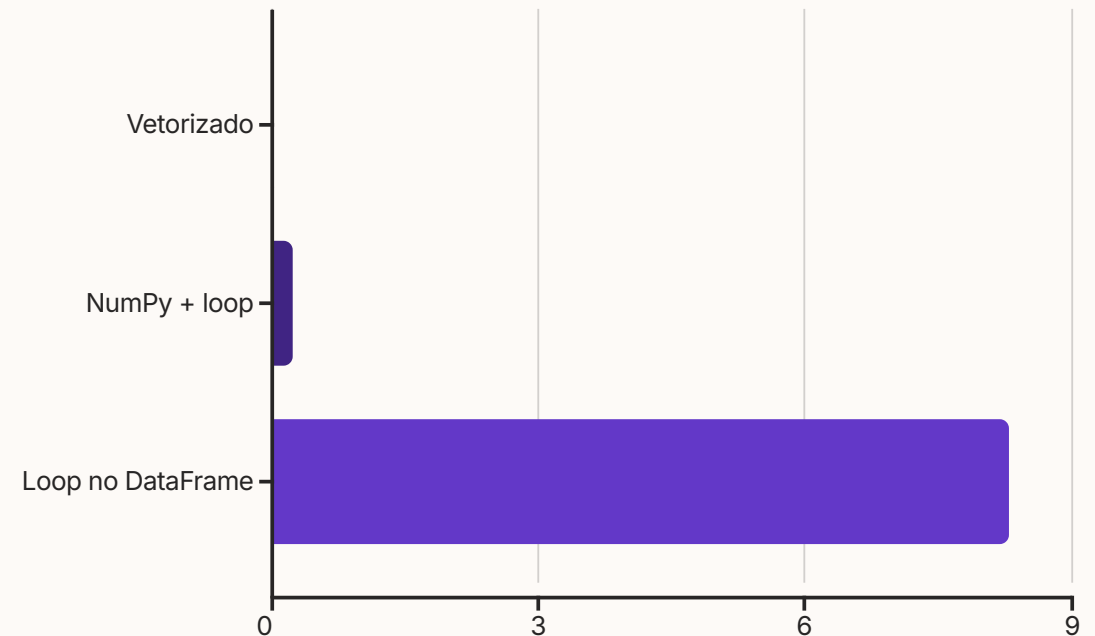
Converter para NumPy e voltar

DataFrames são construídos sobre arrays NumPy. Converter para NumPy permite acesso mais direto e rápido aos valores. Essa abordagem é intermediária: mais rápida que loops no DataFrame, mas mais lenta que vetorização pura.

```
from time import perf_counter
import numpy as np
import pandas as pd

t0 = perf_counter()
hw = pd.DataFrame({"height": rheight, "weight": rweight})
hwm = hw.to_numpy()
bmi = np.empty(len(hwm))
for i in range(len(hwm)):
    bmi[i] = hwm[i, 1] / (hwm[i, 0] ** 2)
hw["bmi"] = bmi
t1 = perf_counter()

print("Tempo NumPy + loop:", t1 - t0)
print(hw.head())
```



Conclusão: A conversão para NumPy melhora o desempenho comparada ao loop direto no DataFrame, mas a vetorização pura continua sendo a melhor escolha.

Referências

Think Python

Downey, A. *Think Python: How to Think Like a Computer Scientist*. O'Reilly Media. Introdução essencial aos fundamentos de programação e pensamento computacional com Python.

Python Data Science Handbook

VanderPlas, J. *Python Data Science Handbook*. O'Reilly Media. Guia abrangente das ferramentas essenciais para trabalhar com dados em Python, incluindo NumPy, Pandas e bibliotecas de visualização.

Data Science from Scratch

Grus, J. *Data Science from Scratch*. O'Reilly Media. Aprenda fundamentos de ciência de dados construindo algoritmos e ferramentas do zero com Python.